Serial versus OpenMP/CUDA Parallel Synthesis of Similarity-Ordered Photo Transition Videos

1st Awwab Wadekar
Dept. of Computer Engineering and Information Technology
Veermata Jijabai Technological Institute (VJTI)
Mumbai, India
aswadekar_b24@it.vjti.ac.in

2nd Taha Valiji
Dept. of Computer Engineering and Information Technology
Veermata Jijabai Technological Institute (VJTI)
Mumbai, India
tmvaliji_b24@it.vjti.ac.in

Mentor: Dr. Sandip Tukaram Shingade
Dept. of Computer Engineering and Information Technology
Veermata Jijabai Technological Institute (VJTI)
Mumbai, India
stshingade@it.vjti.ac.in

***Abstract*—Turning an unordered folder of photographs into a smooth slideshow video requires three costly steps: extracting a descriptor from every image, arranging the images so that neighbours look alike, and synthesising and encoding many intermediate transition frames. We present a complete pipeline for this task and implement it twice from the same source: a single-threaded CPU version and a parallel version that uses OpenMP for feature extraction and similarity computation and CUDA for frame blending. Images are described by 3-D HSV colour histograms, compared with histogram correlation, ordered by a greedy nearest-neighbour tour, and joined by cross-dissolve frames. We benchmark both versions on five class-balanced subsets (100–1000 images) of a public landscape dataset, at four output resolutions (up to 1920×1080) and three transition lengths, giving 120 timed runs on a 16-thread CPU with an RTX 4050 GPU. The parallel stages are substantially faster: feature extraction by $5.7\times$ on average (up to $10.0\times$) and similarity computation and ordering by $7.5\times$ (up to $8.9\times$). The end-to-end speedup, however, is only 1.04–$1.13\times$ (mean $1.11\times$), because writing the video through a sequential software encoder accounts for 91–99% of the serial run time. We quantify this Amdahl-type limit and identify encoder-side parallelism as the necessary next step.**

***Index Terms*—parallel computing, OpenMP, CUDA, image similarity, colour histograms, video generation, frame interpolation, speedup analysis**

I. Introduction

Personal and online photo collections are large and usually unordered. Presenting them as a video, where each photograph dissolves smoothly into the next, is a common request, but it is computationally heavy: every image must be decoded and analysed, every pair of images compared in order to choose a pleasing sequence, and tens of thousands of transition frames rendered and compressed. The work is also naturally parallel in most of its stages, which makes it a good case study for comparing a serial implementation with a CPU–GPU parallel one.

Our design draws on three lines of earlier work. Snavely *et al.* presented a system for browsing large, unstructured photo collections that uses image-based rendering to *smoothly transition between photographs* [1]; we adopt this setting of turning an unordered photo set into a continuous visual sequence. Niklaus *et al.* cast video frame interpolation as a (separable) convolution applied to a pair of input frames, and note that per-pixel 2-D kernels become very memory-hungry at 1080p (26 GB for 41×41 kernels), which motivates their separable, whole-frame formulation [2]. Fan *et al.* (Co-ViA) observed that in learned interpolation one stage—optical-flow estimation, over 60% of run time in the baseline they analyse—dominates, and obtained large gains only by restructuring the pipeline around that bottleneck [3]. We use the first as the application setting, the second as the interpolation viewpoint (in its simplest, learning-free form: a per-pixel blend of two frames), and the third as the methodological lesson that the benefit of accelerating any stage is bounded by the share of the stages that remain unchanged.

Contributions. (i) A complete, reproducible serial and a parallel (OpenMP + CUDA) implementation of an image-to-video transition pipeline sharing identical algorithms and parameters; (ii) a command-line and JSON-driven benchmarking harness that sweeps datasets, resolutions and transition lengths and records per-stage timings; (iii) a 120-run measurement study with a stage-level analysis explaining why large stage speedups translate into a modest end-to-end gain.

Organisation. Section II states the problem. Section III describes the approach and algorithms, Section IV their implementation, and Section V the datasets and experimental procedure. Results are reported in Section VI and analysed in Section VII; Section VIII concludes.

II. Problem Statement

Given a directory of N images $\mathcal{I} = \{I_1, \ldots, I_N\}$ of arbitrary sizes, an output resolution $W \times H$ and a number $F \geq 0$ of intermediate frames per image pair, produce a single video that (a) shows every image exactly once, (b) orders the images so that consecutive images are as visually similar as possible, and

(c) inserts F synthesised frames between consecutive images so that one image fades into the next. The video contains

$$T = (N-1)(F+1) + 1 \tag{1}$$

frames. We want to know:

1) How much faster can each stage be made by CPU multi-threading (OpenMP) and GPU offload (CUDA) compared with an equivalent single-threaded implementation?
2) How do the speedups depend on the dataset size N, the resolution $W \times H$, and the transition length F?
3) Which stage limits the end-to-end speedup, and by how much?

Ordering the images optimally is a travelling-salesman-type problem; we do not seek the optimum but a cheap, deterministic heuristic that is identical in both implementations, so that the comparison isolates the effect of parallelisation.

III. Approach and Algorithms

A. Pipeline overview

The pipeline (Fig. 1) has five stages. Stages 1 and 2 are data-parallel across images and image pairs; Stage 3 is a short sequential scan; Stage 4 is data-parallel across pixels; Stage 5 is a sequential stream into the video encoder.

B. Image descriptor: HSV colour histogram

Each image is resized to the output resolution $W \times H$ (bilinear interpolation) and converted from BGR to HSV, a colour space in which hue is separated from brightness and is therefore more stable under lighting change than raw RGB. A three-dimensional histogram with 16 hue ($[0, 180)$), 8 saturation and 8 value bins ($[0, 256)$ each) is computed, giving a descriptor of $16 \cdot 8 \cdot 8 = 1024$ bins, which is then min–max normalised to $[0, 1]$. Colour-histogram matching is a classical, inexpensive way of comparing images [4]; it is robust to small viewpoint changes, which suits photographs of the same scene type.

C. Pairwise similarity

For histograms H_i, H_j with means $\bar{H}_i, \bar{H}_j$ over the $B = 1024$ bins, similarity is the Pearson correlation

$$s_{ij} = \frac{\sum_b (H_i(b) - \bar{H}_i)(H_j(b) - \bar{H}_j)}{\sqrt{\sum_b (H_i(b) - \bar{H}_i)^2 \; \sum_b (H_j(b) - \bar{H}_j)^2}}, \tag{2}$$

(OpenCV's `HISTCMP_CORREL`). Because $s_{ij} = s_{ji}$, only the upper triangle is computed and mirrored: $N(N-1)/2$ comparisons of cost $O(B)$ each.

D. Ordering

The images are sequenced by the greedy nearest-neighbour rule of Algorithm 1: start at the first image, and repeatedly move to the not-yet-visited image with the highest correlation to the current one. It costs $O(N^2)$, is deterministic, and is identical in the serial and parallel builds. It gives no optimality guarantee, and we do not evaluate the visual quality of the resulting order.

Algorithm 1 Greedy similarity ordering

Input: similarity matrix $S \in \mathbb{R}^{N \times N}$
Output: permutation π of $\{1, \ldots, N\}$

$\pi \leftarrow [1]$; *visited*$[1] \leftarrow$ **true**; $c \leftarrow 1$
for $k = 2$ **to** N **do**
  $n \leftarrow \arg\max_{j:\, \neg \textit{visited}[j]} S_{cj}$
  append n to π; *visited*$[n] \leftarrow$ **true**; $c \leftarrow n$
end for
return π

TABLE I
Mapping of stages to execution resources

Stage	Work items	Serial build	Parallel build
1 Features	N images	loop	OpenMP, dynamic
2 Similarity	$N(N-1)/2$ pairs	loop	OpenMP, dynamic
3 Ordering	N steps	loop	loop (same)
4 Blend	$W \cdot H \cdot 3$ bytes/frame	CPU loop	CUDA kernel
5 Encode	T frames	sequential	sequential

E. Transition synthesis

For consecutive images $A = I_{\pi_k}$ and $B = I_{\pi_{k+1}}$ the f-th intermediate frame ($f = 1, \ldots, F$) is the cross-dissolve

$$I_f(x, y, c) = \big\lfloor (1 - \alpha_f)\, A(x, y, c) + \alpha_f\, B(x, y, c) \big\rfloor, \tag{3}$$

with $\alpha_f = f/(F+1)$, for every pixel (x, y) and channel c. This is the simplest instance of the "combine two frames with a per-pixel weighting" idea behind convolution-based interpolation [2], with a constant, content-independent kernel and no motion estimation; it therefore handles the unrelated pairs in a photo collection (which have no meaningful motion) at a cost of two multiplies and one add per byte. Each pixel and channel is independent, which makes the stage embarrassingly parallel. Frames are written in order, A first and the last image at the end, giving Eq. (1) frames at 60 fps.

F. Parallelisation strategy

Table I summarises how each stage is mapped. Stages 1 and 2 use *data parallelism with dynamic scheduling*: image decode times differ, and the triangular loop in Stage 2 gives early rows more work than late rows. Stage 4 uses one GPU thread per pixel. Stage 5 cannot be parallelised across frames with the software encoder used here, because frames must be appended in order.

IV. Implementation

A. Code organisation and build

The code base is C++17 with CUDA. Both executables are built by one CMake project (Release configuration by default, so that timings reflect optimised code):

- `pipeline_parallel`: `main.cpp` + `kernels.cu`, linked against OpenCV, OpenMP and the CUDA runtime.
- `pipeline_serial`: `main_serial.cpp`, linked against OpenCV only; it contains no OpenMP pragma and no CUDA call, so it is a genuine single-threaded baseline.

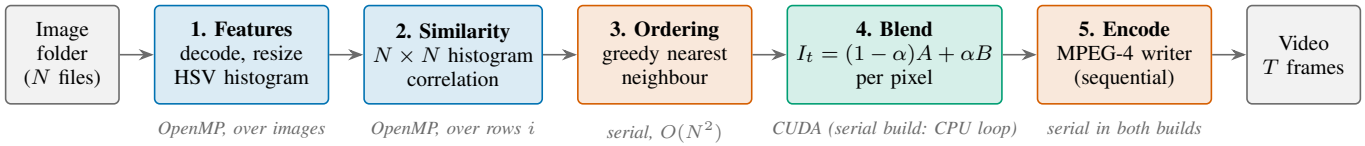


Fig. 1. Pipeline stages and where parallelism is applied. Blue: parallelised on the CPU; green: parallelised on the GPU; orange: sequential in both builds. In the serial build every stage runs single-threaded on the CPU.

Listing 1. CUDA blend kernel (Eq. (3)).

```
__global__ void blendKernel(const unsigned char*
    __restrict__ A,
                    const unsigned char* __restrict__ B,
                    unsigned char* __restrict__ out,
                    float alpha, int totalPixels) {
  int idx = blockDim.x * blockIdx.x + threadIdx.x;
  if (idx < totalPixels) {
    int p = idx * 3; // B, G, R
    float inv = 1.0f - alpha;
    out[p+0] = (unsigned char)(A[p+0]*inv + B[p+0]*alpha);
    out[p+1] = (unsigned char)(A[p+1]*inv + B[p+1]*alpha);
    out[p+2] = (unsigned char)(A[p+2]*inv + B[p+2]*alpha);
  }
}
```

A shared header, `common.hpp`, contains everything that must be identical in both builds—argument parsing, image listing, the HSV histogram, and the greedy ordering of Algorithm 1—so differences in run time come only from how the stages are executed.

B. Command-line interface

Both programs take named, order-independent options: `-dataset`, `-width`/`-height` (accepted only together; default 1280×720), `-frames` (default 30), `-output`, and `-metrics`, which writes the stage timings as JSON. Invalid values (non-positive sizes, only one of width/height, unknown or positional arguments) are rejected with exit code 2.

C. Parallel stages

Stage 1 is a single `#pragma omp parallel for schedule(dynamic)` over the image list: each thread decodes, resizes and histograms its own images and writes only to its own slots of the pre-allocated arrays, so no locking is needed. Stage 2 uses the same pragma over the row index i of the upper triangle, each iteration computing s_{ij} for $j > i$ and writing both S_{ij} and S_{ji} (distinct elements, hence race-free). Images that fail to decode are dropped before Stage 2. The ordering (Stage 3) runs on one thread in both builds.

D. GPU blending

For each consecutive pair the two images are copied to the device once, and each of the F intermediate frames is produced by a kernel launch followed by a device-to-host copy into a reusable frame buffer that is handed to the encoder. The kernel assigns one thread to one pixel (three bytes), with 256 threads per block (Listing 1); the host wrapper synchronises after each launch.

The serial build uses the same expression in a plain loop over the $W \cdot H \cdot 3$ bytes of the frame, with the same truncation.

E. Video encoding

Both builds stream frames directly into an OpenCV `VideoWriter` using the MPEG-4 Part 2 (`mp4v`) codec at 60 fps; no frame is stored beyond the one being encoded. The reported "blend + encode" time therefore covers blending (CPU or GPU, including host–device transfers), codec work and file output. All N resized images are held in memory, which costs $N \cdot W \cdot H \cdot 3$ bytes (6.2 GB for N=1000 at 1920×1080).

F. Benchmark driver

A Python driver (`run_pipeline.py`) configures and builds the project with CMake, then reads a JSON file describing datasets, each with a path, an output path, a list of (W, H) pairs and a list of F values. For every dataset and every combination it runs the serial and then the parallel executable. The driver validates its input and *reports but does not abort on* problems: a missing path, a dataset with fewer than 20 images, or an invalid setting (e.g. a negative height) causes only that dataset or setting to be skipped. Every run's three stage times, total frames and wall time are appended immediately to a CSV file, so a long sweep survives interruptions. To limit disk usage the driver deletes each run's video after its timings are saved, keeping only the 1920×1080, F=20 video of each dataset.

V. Dataset and Processing

A. Source data

We use the public Kaggle dataset "Landscape Recognition | Image Dataset (12k Images)" [9], a multi-class landscape classification set. Its training split has five classes—Coast, Desert, Forest, Glacier and Mountain—of 2000 JPEG images each. The source images are small (for example 275×183 and 300×168), so every output resolution up-samples them; this affects image content but not the amount of computation per pixel, which is what we measure.

B. Subsets

A setup script samples, with a fixed random seed (42), an equal number of images from each class and copies them into flat folders, producing five datasets of $N = 100, 200, 500, 750$ and 1000 images (Table II). Class balance ensures that the similarity structure is non-trivial: images of the same class tend to correlate more strongly than images of different classes.

C. Experimental procedure and platform

Every dataset is run with every resolution and every F in both modes (120 runs, all completed successfully). The driver is executed once, sequentially, so serial and parallel

TABLE II
DATASETS AND EXPERIMENTAL GRID

Dataset	Images N	Per class	Pairs $N(N-1)/2$
`landscape_100`	100	20	4,950
`landscape_200`	200	40	19,900
`landscape_500`	500	100	124,750
`landscape_750`	750	150	280,875
`landscape_1000`	1000	200	499,500

Resolutions: 640×360, 1024×768, 1280×720, 1920×1080. Transition lengths $F \in \{10, 20, 30\}$. Modes: serial, parallel. Total: $5 \times 4 \times 3 \times 2 = 120$ runs.

runs never compete for the machine. Timings are taken inside the executables with a monotonic clock and cover three stages: *feature extraction* (Stage 1, including image decoding), *similarity and ordering* (Stages 2–3), and *blend + encode* (Stages 4–5); the *pipeline time* is their sum and excludes process start-up and file listing. The platform is a laptop with an Intel Core i5-13450HX CPU (10 cores, 16 hardware threads), 31 GB RAM, and an NVIDIA GeForce RTX 4050 Laptop GPU; software is Linux, GCC 16.2, OpenCV 5.0 [5], OpenMP [6] with the default of 16 threads, and CUDA 13.4 [7].

D. Output consistency

In a preliminary check on a separate 233-image folder at 640×360 with F=2, both builds produced videos with the same number of frames (697, as predicted by Eq. (1)); after lossy MPEG-4 encoding, decoded frames differed by a mean absolute value of 0.52 intensity levels (out of 255; largest per-frame mean 1.41). We therefore regard the two outputs as visually equivalent, and did not otherwise evaluate image quality.

VI. RESULTS

A. Stage-level times

Table III gives the per-stage times of both builds at 1920×1080 and F=20 for every dataset. Fig. 2 shows the same breakdown graphically at three resolutions: in both builds the blend-and-encode stage is almost the entire bar.

B. Parallel-stage speedups

Fig. 3 shows the speedup of the two parallel CPU stages. Over all 120 configurations, feature extraction is accelerated by 4.3–10.0× (mean 5.7×) and similarity plus ordering by 4.2–8.9× (mean 7.5×), using 16 threads. Feature extraction is fastest at the lowest resolution (mean 6.9× at 640×360, versus 5.1–5.5× at the other resolutions; Table IV). The similarity stage improves with N (mean 5.7× at N=100, 7.4× at 200, and 8.0–8.4× for $N \geq 500$).

C. End-to-end results

Fig. 4 (a) plots the pipeline time against the number of frames written, on logarithmic axes: all resolutions follow straight lines of unit slope, and parallel runs lie just below serial runs. Fig. 4 (b) shows that the end-to-end speedup is between 1.04× and 1.13× (mean 1.11×), almost independent of N beyond N=200 and slightly lower at 640×360. Across the whole study, the serial runs total 0.99 h of pipeline time and the parallel runs 0.88 h. The longest single run (N=1000, 1920×1080, F=30, 30,970 frames) took 335 s serially and 298 s in parallel.

D. Effect of transition length

Table V varies F for N=500 at 1280×720. Pipeline time grows linearly with the number of frames, and the speedup is the same for all three settings.

VII. ANALYSIS

A. The parallel stages scale, but sub-linearly

The OpenMP stages run 5.7× (features) and 7.5× (similarity) faster on 16 threads. Neither approaches the thread count. We do not have profiling data to attribute this precisely; plausible contributors are that the CPU has 6 performance and 4 efficiency cores rather than 16 equal ones (so 16 threads are not 16 full cores), that decoding and resizing are memory-bandwidth-bound, and, for the similarity stage, that the triangular iteration space leaves some imbalance and that the stage is short (4–400 ms serial) and therefore sensitive to start-up cost at small N. The latter is consistent with Fig. 3(b), where the speedup rises with N and then flattens near 8×. Per image, feature extraction costs 0.96 ms (serial) versus 0.14 ms (parallel) at 640×360 and 4.7 ms versus 0.92 ms at 1920×1080; the larger fixed share of JPEG decoding at low resolution (the source images are tiny and identical across settings) is a possible reason for the better scaling there, but we have not isolated it.

B. Encoding dominates, which caps the end-to-end gain

Blend and encode takes 91–99% of the serial pipeline time (mean 96.9%), and 98.8–99.7% of the parallel pipeline time (Fig. 2; Table IV). Its cost is proportional to the number of frames (Fig. 4a; Table V) and grows with the frame area: for the parallel build the stage costs 1.3 ms per frame at 640×360, 4.0 at 1024×768, 4.7 at 1280×720 and 10.4 ms at 1920×1080. By Amdahl's law [8], speeding up a fraction p of the run time by a factor s gives an overall speedup $1/\big((1-p)+p/s\big)$, bounded by $1/(1-p)$. Here Stages 1–3 make up only $p \approx$ 1–9% of the serial time; accelerating them alone would give a speedup of 1.01–1.08× (mean 1.03×, dashed line in Fig. 4b), however fast they became. This is the same structure that Fan *et al.* exploit in the opposite direction: they identify the dominant stage of their pipeline and redesign around it [3]; in our pipeline the dominant stage is video encoding, not feature analysis.

C. GPU blending gives a small additional gain

The observed mean end-to-end speedup, 1.11×, is larger than the 1.03× explained by Stages 1–3, because the blend-and-encode stage itself is 1.08× faster on average (0.99–1.12×) when the blend runs on the GPU. At 1920×1080 it falls from 11.4 to 10.4 ms per frame, i.e. the GPU saves about 1 ms per frame net of the ≈ 6 MB device-to-host copy that each frame requires. This is an inference from the totals; we did

TABLE III
STAGE TIMES (MS), SERIAL (S) VERSUS PARALLEL (P), AT 1920 × 1080, $F = 20$. SPEEDUP = PIPELINE S / PIPELINE P.

		Feature extraction		Similarity + order		Blend + encode		Pipeline total		
N	Frames T	S	P	S	P	S	P	S	P	Speedup
100	2080	481	98	4	1	23,560	21,366	24,045	21,465	1.12
200	4180	938	211	16	2	47,310	43,586	48,264	43,799	1.10
500	10480	2,394	418	100	12	116,750	106,726	119,243	107,156	1.11
750	15730	3,437	654	230	26	176,329	162,063	179,997	162,743	1.11
1000	20980	4,499	955	397	48	235,849	213,825	240,745	214,828	1.12

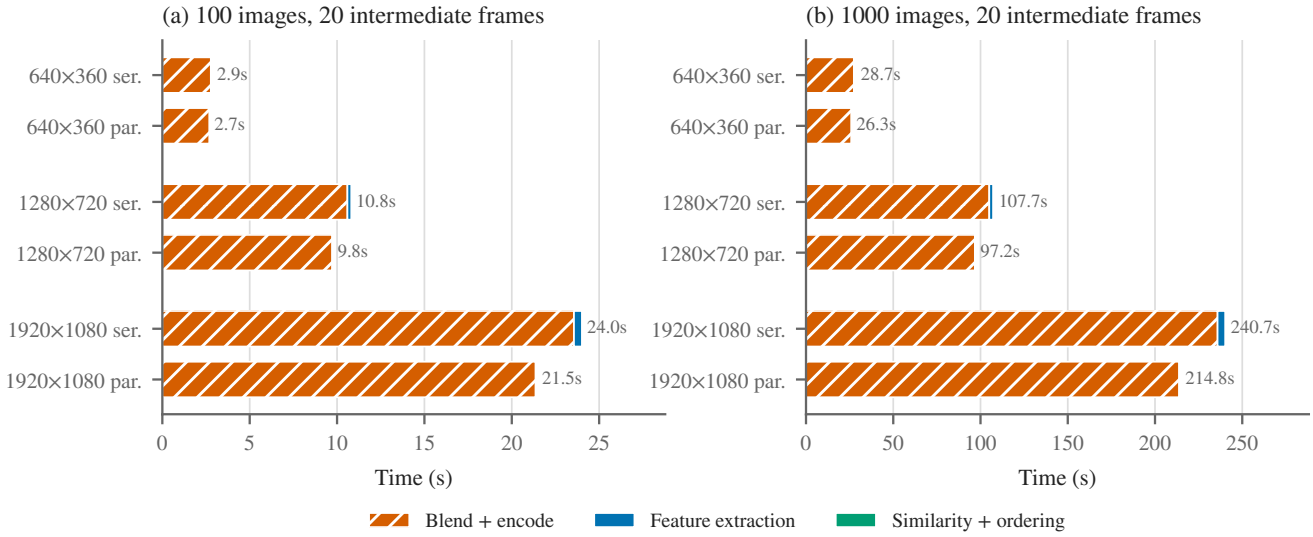


Fig. 2. Time breakdown of the serial (ser.) and parallel (par.) builds for N=100 (a) and N=1000 (b) at three resolutions, F=20. Totals are annotated. Feature extraction and similarity/ordering (blue and green) are barely visible next to blend + encode (hatched orange).

TABLE IV
MEAN SPEEDUP (S/P) BY OUTPUT RESOLUTION OVER ALL N AND F (15 CONFIGURATIONS EACH). "ENC. %" IS THE BLEND+ENCODE SHARE OF THE SERIAL PIPELINE TIME.

Resolution	Feat.	Sim.+ord.	Blend+enc.	Total	Enc. %
640×360	6.95	7.53	1.04	1.09	95.3
1280×720	5.46	7.39	1.09	1.11	97.3
1920×1080	5.11	7.78	1.10	1.12	97.7
1024×768	5.24	7.44	1.08	1.11	97.2

TABLE V
EFFECT OF THE NUMBER OF INTERMEDIATE FRAMES F (N=500, 1280 × 720). S/P: SERIAL/PARALLEL PIPELINE TIME.

F	Frames T	S (ms)	P (ms)	Speedup	P enc. ms/frame
10	5490	31,031	27,752	1.12	5.01
20	10480	53,888	48,269	1.12	4.58
30	15470	74,905	66,619	1.12	4.29

not time blending separately. It shows that the per-pixel blend is cheap even on the CPU compared with encoding a frame, so offloading it cannot change the picture by more than about 10%. The GPU gain is smallest at 640 × 360, where the blend work per frame is smallest and the fixed per-frame launch and transfer overheads are largest.

D. Longer transitions are slightly cheaper per frame

Table V shows the encode cost per frame falling from 5.0 to 4.6 to 4.3 ms as F increases from 10 to 30, while total time remains proportional to T. Consecutive blended frames differ only slightly, and an inter-frame video codec compresses such frames well, whereas the first frame of each pair is a new image; we did not verify this with codec statistics.

E. Threats to validity

Each configuration was run once, so run-to-run variance is not quantified; the smoothness of the curves (Figs. 3 and 4) suggests it is small compared with the effects discussed, but the individual stage numbers for the shortest stages should be read with care. All results come from one laptop whose clocks may vary with temperature, and from one codec (`mp4v`) selected through OpenCV; a hardware encoder or a multi-threaded encoder would change the balance between stages. We did not measure ordering quality or interpolation quality (e.g. PSNR), and the cross-dissolve does not model motion, so these results speak to execution time only.

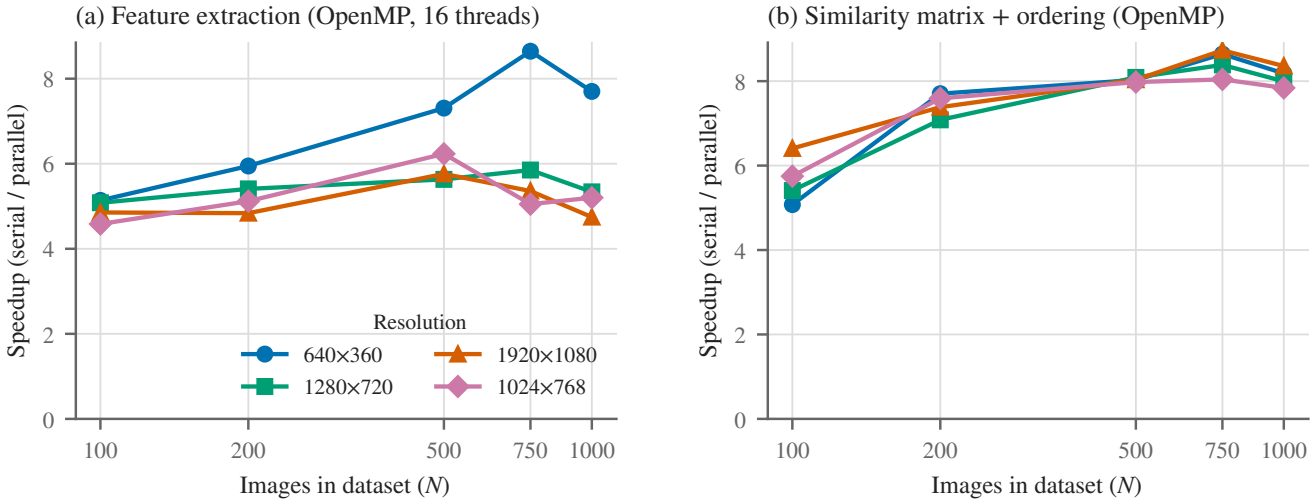


Fig. 3. Speedup of the OpenMP stages (serial time divided by parallel time) as a function of dataset size, for each output resolution (means over $F \in \{10, 20, 30\}$).

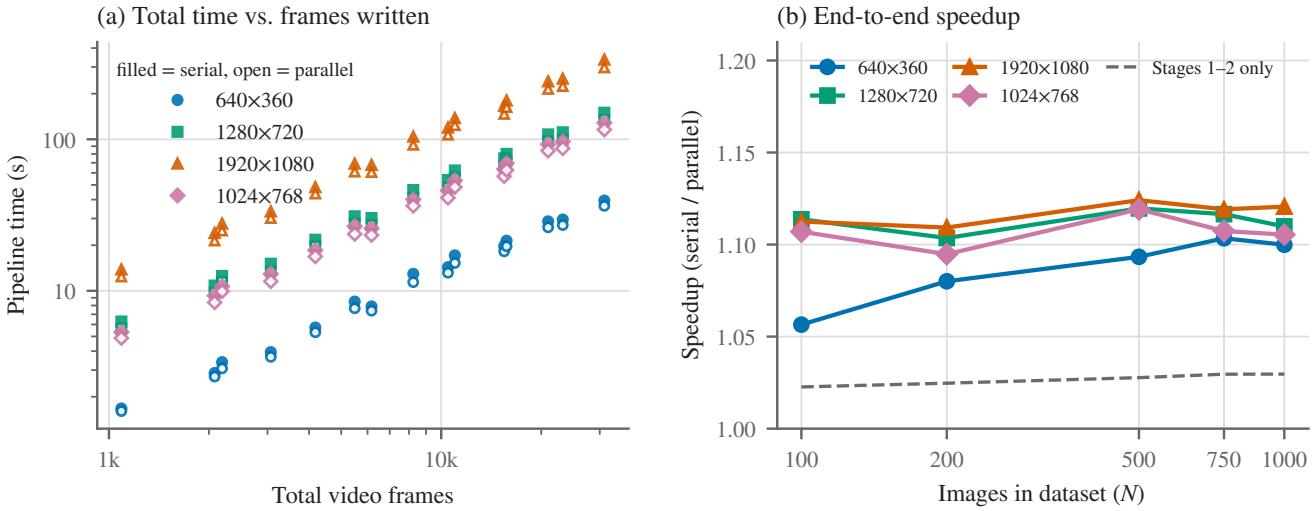


Fig. 4. (a) Pipeline time versus number of frames written (log–log). Filled markers: serial; open markers: parallel. (b) End-to-end speedup versus dataset size (means over F). The dashed line is the speedup that would result from accelerating only Stages 1–3 and leaving blend + encode at its serial time.

VIII. Conclusion and Future Work

We implemented a photo-to-video transition pipeline in a serial and an OpenMP/CUDA form and measured both over 120 configurations. The parallel stages achieve $5.7\times$ (feature extraction) and $7.5\times$ (similarity and ordering) mean speedups, yet the whole pipeline is only $1.11\times$ faster on average, because 91–99% of the serial time is spent blending and sequentially encoding frames. Further CPU or GPU work on the analysis stages cannot change this; the next gains must come from the encoder side. Directions we consider most promising are: (i) hardware video encoding (e.g. NVENC) or a multi-threaded encoder; (ii) overlapping blending with encoding through a producer–consumer queue and CUDA streams, in the spirit of the tile- and stage-level pipelining of Co-ViA [3]; (iii) writing independent video segments in parallel and concatenating them; (iv) a better ordering heuristic (e.g. 2-opt improvement), whose quality should be measured; and (v) replacing the cross-dissolve with a learned interpolation such as adaptive separable convolution [2], which would raise the compute per frame and might shift the balance back toward the GPU.

References

[1] N. Snavely, S. M. Seitz, and R. Szeliski, "Photo tourism: Exploring photo collections in 3D," *ACM Trans. Graphics (Proc. SIGGRAPH)*, vol. 25, no. 3, 2006.

[2] S. Niklaus, L. Mai, and F. Liu, "Video frame interpolation via adaptive separable convolution," in *Proc. IEEE Int. Conf. Computer Vision (ICCV)*, 2017, arXiv:1708.01692.

[3] H. Fan, Q. Sun, J. Wu, W. Lu, X. Li, and G. Yan, "Co-ViA: A video frame interpolation accelerator exploiting codec information reuse," in *Proc. 61st ACM/IEEE Design Automation Conf. (DAC)*, San Francisco, CA, USA, 2024, doi:10.1145/3649329.3655669.

[4] M. J. Swain and D. H. Ballard, "Color indexing," *Int. J. Computer Vision*, vol. 7, no. 1, pp. 11–32, 1991.

[5] G. Bradski, “The OpenCV library,” *Dr. Dobb’s Journal of Software Tools*, 2000.

[6] L. Dagum and R. Menon, “OpenMP: An industry standard API for shared-memory programming,” *IEEE Computational Science and Engineering*, vol. 5, no. 1, pp. 46–55, 1998.

[7] J. Nickolls, I. Buck, M. Garland, and K. Skadron, “Scalable parallel programming with CUDA,” *ACM Queue*, vol. 6, no. 2, pp. 40–53, 2008.

[8] G. M. Amdahl, “Validity of the single processor approach to achieving large scale computing capabilities,” in *Proc. AFIPS Spring Joint Computer Conf.*, 1967, pp. 483–485.

[9] Kaggle user `utkarshsaxenadn`, “Landscape Recognition | Image Dataset (12k Images),” Kaggle. [Online]. Available: https://www.kaggle.com/datasets/utkarshsaxenadn/landscape-recognition-image-dataset-12k-images